React JS

It is library to develop the frontend of a web application. Using React JS we can develop SPA. React JS is developed by facebook.

Old approach: html, css, js

Home.html: navbar: 30 lines html code footer: 50 line code

About.html: navbar: 30 line html code footer: 50 line code

Admissions.html: navbar : 30 line html code footer: 50 line code

50 html files

drawbacks:

1. No html code reusability
2. Not easy to maintain, not easy to scale
3. When user is visiting any of the page, the browser is getting a reload and due to this reload everytime browser will repaint the entire page. This is not efficient approach. Performance is the bottleneck.

SPA : Single Page Application

SPA will only have a single html file, and entire application will be displayed on that single html file only. When users are navigation between the links the browser will not be reloaded, and on the basis of url a new content will be dynamically displayed on that html page.

For SPA we have to break down the entire frontend into **components**.

Instead of creating multiple **pages**, we now have to create **components.**

Component: Ui element visible on the screen. Eg: button, input field, text, heading, table, link, image etc . Component can be at atomic level and the complete screen can also be referred as component.

For creating a component we have to create a **.js / .jsx** file

How to create a react application?

We will make use of a tool i.e. **vite** to generate/create a react application.

use this command to create a react app:

```
npx create-vite@latest
```

```
npm run dev
```

React application will have one already created component i.e. **App component** i.e in the src there might be a file **App.js / App.jsx. App** component is getting displayed first.

Entire react application gets displayed on the root level **index.html.**

How to create a component ?

Types of components:

1. Class component: To create this component we have to make use of the class syntax. Now a class is going to define the behavior or functionality of that component. We can make use of OOPs principles to create this type component. From the beginning of react js class components are there.
2. Function component: To represent the component if a function has been created then it is a function component. Function components are the light weight components.

Creating a class component:

Navbar.jsx:

```
import {Component} from "react";

export class Navbar extends Component{

        render(){
        // to provide something
        // render() will provide or display the ui for the cl
ass component
        return (
            <div>
```

```
                <a href="#">Home</a>
                <a href="#">About Us</a>
                <a href="#">Contact Us</a>
            </div>
        )
    }
}
// by this above inheritance we can say Navbar class will hav
e all the features to get recogniz as a ui component
```

Note: To make use of any component (either it is class component or function component) or we if we need to display that component we will get the tag name corresponding to the component name. Every tag will have opening and closing, either it is predefined html tag or it is custom tag for the component.

JSX: JavaScript and XML: .jsx extension is enabling a developer to write js code along with the html code.

Creating function component:

NavigationBar.jsx

```
export function NavigationBar(){
    return (
        <div>
            <a>..</a>
        </div>
    )
}
```

Note: JSX expressions must have one parent element

Features of a component:

1. Prop
2. State

These two features are going to be there in each and every type of component

Props: attributes/properties which are passed in the component

```
<p align="center">Hello</p>
<img src="" width="" height="">
<a href=""></a>
```

Props in a function component:

```
export function Intro(props) {

    const { headingText, description, buttonText } = props;

    return (
        <div>
            <h1>{headingText}</h1>
            <p>{description}</p>
            <button>{buttonText}</button>
        </div>
    )
}
```

```
<Intro headingText="" description="" buttonText="" ></Intro>
```

Note: React js will first prepare an object of all the props passed in the component

eg:

```
{
            headingText: "Welcome to our app",
            description: "This is our first react app",
            buttonText: "Click me"
          }
```

and the same object will be passed as argument of the function component.

The argument of function component will always receive the props only

A component should not change the value of its props, and it also cannot decide the value of its own props.

Props can be used to pass data from one component to another component.

Props in a class component:

For a class component props will become its property and we already have got a property from Component class due to inheritance and the name of that property is **“props”. Props now in a class are the member of a class**

```
this.props = {
            headingText: "Welcome to our app",
            description: "This is our first react app",
            buttonText: "Click me"
          }
```

```
export class Introduction extends Component{

    render(){
        console.log(this.props);
        const {headingText, description, buttonText} =  this.
props;
        return (
            <div>
                <h1>{headingText}</h1>
                <p>{description}</p>
                <button>{buttonText}</button>
            </div>
        )
    }
}
```

```
<Introduction headingText="" description="" buttonText="" ></
Intro>
```

Receiving props in:

1. Function component: **argument**
2. Class component: **class level member with inherited property i.e. this.props**

State:

State is such a type of property which a component can create by its own, can decide its value, and can change its value also

State is a native feature of a class component.

State in a class component:

state is class level member/property that is inherited from the Component class. Inside of the constructor we have to initialize it. We have to refer state on this keyword: this.state

```
import { Component } from "react";

export class Scoreboard extends Component{

    constructor(){
        super();
        this.state = {
            score : 0
        }
    }

    increaseScore = ()=>{
        this.setState({score: this.state.score + 3 });
    }

    decreaseScore = ()=>{
       this.setState({score: this.state.score - 1 });
    }

    render(){
        return (
```

```
            <div>
                <h1>Score: {this.state.score} </h1>
                <button onClick={this.increaseScore}>+</butto
n>
                <button onClick={this.decreaseScore}>-</butto
n>
            </div>
        )
    }

}
```

When state is updated/changed the component will be re-rendered

Virtual DOM: Copy of the actual browser DOM

Virtual DOM is faster

Virtual DOM → translated → Actual DOM

State in a function Component:

State is not a native feature of a function component.

Note: Not only the state, we can have certain more feature those were native to the class components only. Now in order to implement same feature in a function component react is providing the hooks.

Hook: hook is just a function.

Hook function naming convention: suggestion is that we should always have “use” prefix for the hook function name

To create a state in function component make use of: **useState()**

```
useState(initial-value); // will return two things
// 1. state value/data
// 2. function to update this state
// these two things will be returned in the form of array
const rs = useState(0); // rs is array
// rs[0] state value, data
// rs[1] function to update this state
```

```
// destructure the array returned by useState()
const [x,y] = useState(0); // x = state value, data & y= func
tion to update this state

// x and y are not readable names we must give a readable nam
e

const [score, setScore] = useState(0);
const [player, setPlayer] = useState("");
const [users, setUsers] = useState([]);
```

```
import { useState } from "react"

export function ScoreCard() {

    const [score, setScore] = useState(0);

    const increaseScore = () => {
        setScore(score + 3);
    }

    const decreaseScore = () => {
        setScore(score - 1);
    }

    return (
        <div>
            <h1>Score : {score}</h1>
            <button onClick={increaseScore}>+</button>
            <button onClick={decreaseScore}>-</button>
        </div>
    )
}
```

Lifecycle of a component:

There are certain functions already defined and they are going to be called automatically for each lifecycle stage of a component. Lifecycle functions are natively available in a class components, but for function component we need to make use of a hook.

Mounting phase : component is going to display on the screen

Unmounting phase: component is getting removed from the screen

Lifecycle in a class component:

constructor() : when a component will be mounted, first of all constructor will be called. It will be called only a single time during the entire lifecycle.

render(): When cons has finished the job, render will be called for the 1st time and it will provide the ui for that component. Now later on, if any state has been updated then **render()** will be called again.

componentDidMount(): It will be called only and only just after the first render call. Now if later render() function is getting executed due to state update then **componentDidMount() will not be called.** In general we can make use this lifecycle function to fetch data at the time of component load.

componentDidUpdate(): it will be called after the state update of the component. When state will be updated, render function will be called again and just after the re-render this will be called.

componentWillUnmount(): it will be called just before the component is getting destroyed. We can perform some clean up jobs in this function.

componentDidCatch(): ErrorBoundary: it will be called whenever there is an error in the component, so we can handle the errors gracefully with the help of this function.

Rendering list in react component:

```
this.state = {
    myList : [10,20,30,40,50]
}
```

```
/*
    [<p>10</p>, <p>20</p>, ... ]
*/
render(){
    return (
        <div>
            {
                this.state.myList.map((value)=>{
                    return (
                        <p>{value}</p>
                    )
                })
            }
        </div>
    )
}

this.state = {
    myList = [
                        {id:111,name:"Arun"},
                        {id:112, name:"Aman"},
                        {id:113, name:"Suraj"}
    ]
}
/*
    [<div>
                        <h4>111</h4>
                        <p>Arun</p>
           </div>,
           <div>
                        <h4>111</h4>
                        <p>Arun</p>
           </div>,
           <div>
                        <h4>111</h4>
                        <p>Arun</p>
```

```
            </div>]
*/
render(){
    return (
        <div>

            {
                this.state.myList.map((value)=>{
                        return (
                            <div>
                                <h4>{value.id}</h4>
                                <p>{value.name}</p>
                            </div>
                        )
                })
            }
        </div>
    )
}
```

Lifecycle in a function component:

Lifecycle functions is a native feature of a class component, but not available directly in a function component. We can make use of a hook to have the lifecycle of a function component.

useEffect(): with this hook we can implement the lifecycle in a function component.

For every lifecycle phase this useEffect() will work

useEffect as componentDidMount:

`useEffect(callback-function, dependency-array);`

```
useEffect(()=>{

}, []); // leave this dependency array as empty to have the f
unctionality of componentDidMount
```

```
// The above useEffect will be called just after the first re
nder of the function component
```

```
import { useEffect} from "react"

export function UsersData(){

    // const [users, setUsers] = useState([]);

    useEffect(()=>{
        fetch("https://jsonplaceholder.typicode.com/users")
        .then((response)=>{
           return response.json()
        })
        .then((usersData)=>{
            console.log(usersData);

        })
        .catch((error)=>{
            console.log(error);
        })
    },[]); // componentDidMount

    return (
        <div>
            <h2>List of Users</h2>
        </div>
    )
}
```

useEffect as componentDidUpdate():

```
useEffect(()=>{}, [dependencies])

// for eg:
```

```
const [x, setX] = useState(0)
const [y, setY] = useState(0)
const [z, setZ] = useState(0)

useEffect(()=>{

},[y,z]);
```

```
import { useEffect, useState} from "react"

export function UsersData(){

    // const [users, setUsers] = useState([]);
    const [x, setX] = useState(100);
    const [y, setY] = useState(500);

    // for the first time all the useEffects are going to be
executed

    useEffect(()=>{
        console.log("did mount...called just after 1st rende
r");

    },[]); // componentDidMount

     useEffect(()=>{
        console.log("did update...called after x or y updat
e");
    },[x,y]); // componentDidUpdate

    function changeX(){
        setX(x+5);
    }
```

```
    function changeY(){
        setY(y+10);
    }

    return (
        <div>
            <h2>List of Users</h2>
            <h2>x={x}, y={y}</h2>
            <button onClick={changeX}>change x</button>
            <button onClick={changeY}>change y</button>
        </div>
    )
}
```

Note: For the first render i.e. during mounting phase the both useEffects will be called. but later on on the state update only the useEffect with dependencies will be called.

useEffect as componentWillUnmount:

```
useEffect(()=>{

    return ()=>{
        // cleanup
    }

}, []); // into any existing useEffects we can return a funct
ion
```

```
componentDidMount -> useEffect(()=>{},[]);
componentDidUpdate -> useEFfect(()=>{}, [dependencies]);
componentWillUnmount -> useEffect(()=>{
    return ()=>{
```

```
    }
}, []);
```

Note: useEffect must not return anything besides a function, which is used for clean-up. You returned: 7

Rule: Every element of the list must have a key prop. React uses that key prop to uniquely identify the elements inside of a list.

Without key:

Virtual DOM: even virtual dom is there and that difference mechanism is going to work but not for the list, if there is any change in the list then entire list is going to be recreated

e1 e2 e3 e4 e5

e1 e2 e6 e4 e5

With key:

Virtual DOM now is going to compare the list for previous dom and the latest dom and now it will not re-create entire list, simply it will update that specific element

Performance optimization:

Memoization is a way of improving the performance of a react application

352 * 922 = **324544 2-3 minutes**

Whenever there is an expensive calculation, lets say it has taken a large amount of time for performing that job and multiple times we have to do that operation.

Without Memoization: regardless of the input is same or different, to get the result the react app will always re-calculate/ re-perform that job.

WIth Memoization: now if input is same for that calculation, it will not re-calculate and simply it will return the result from the caching/cached data. For the 1st time only do the calculation and cache the result, from 2nd time onwards if the same input has been given then return the cached data.

Using memoization we can also stop the unnecessary re-rendering of the component.

When parent component's state will be updated, then along with the parent the child component will also be re-rendered even when the props of child haven't been changed.

To stop the above unneessary re-rendering we have to make use of function **memo()**

memo(component): It will return the memoized version of the component. Now whenever a parent component has been updated, before re-rendering of the child it is going to check for the props that are props same or they are different. If props haven't changed so no re-rendering of the child will be there. A child will only be re-rendered when the props for child has been changed.

```
import { memo } from "react";

export const Greeting = memo((props) => {

    console.log("Child component rendered....");

    const { name } = props;

    return (
        <div>
            <h4>Hello, {name}</h4>
        </div>
    )
})
```

useMemo(): To memoize the result/value of any calculation/operation/logic useMemo will be preferred.

```
import { useMemo, useState } from "react"

export function ExpensiveCalculation() {

    const [x, setX] = useState(10);
    const [y, setY] = useState(50);
```

```
    const optimizedTotal = useMemo(() => {
        let total = 0;
        for (var i = 1; i <= 1000000000; i++) {
            total = total + x;
        }
        return total;
    }, [x]);


    const changeX = () => {
        setX(x + 5);
    }

    const changeY = () => {
        setY(y + 3);
    }

    return (
        <div>
            <h3>x= {x}, y={y}</h3>
            <h3>Total = {optimizedTotal}</h3>

            <button onClick={changeX}>change x</button>
            <button onClick={changeY}>change y</button>

        </div>
    )
}
```

useCallback(): It is used to memoize the functions. Whenever we pass functions as props in the child then even memo() is not going to stop the re-rendering, because for every render of the component a new reference of function will

always be created , memo() will compare the reference of the functions passed and always a memo in the child will get the difference and it will be re-rendered.

```
// when it comes to the functions, then on every render funct
ions will get a new reference
    // even if the child component is memoized then memo func
tion will also not work
    // beacuse in all the renders the function reference will
be different
    // when we pass a function in the props then reference is
compared not the value
    // hence the result is that the memo function will always
get the difference in the prop
```

useCallback prevents the new reference generation if dependency haven't changed yet

```
import { useCallback, useState } from "react"
import { CounterChild } from "./CounterChild";

export function Counter(){
    // parent component

    const [count, setCount] = useState(0);
    const [message, setMessage] = useState("hello");

    const increaseCount = useCallback(() => {
        setCount(count + 10);
    }, [count]);


    const changeMessage = ()=>{
        setMessage(message+" bye");
    }

    /*
```

```
    () => {
        setCount(0 + 10);
    }
    */
    // when it comes to the functions, then on every render f
unctions will get a new reference
    // even if the child component is memoized then memo func
tion will also not work
    // beacuse in all the renders the function reference will
be different
    // when we pass a function in the props then reference is
compared not the value
    // hence the result is that the memo function will always
get the difference in the prop

    return (
        <div>
            <h2>Count = {count}</h2>
            <h2>Message: {message}</h2>
            <button onClick={changeMessage}>Change Message</b
utton>
            <CounterChild plusClickHandler={increaseCount} />
        </div>
    )
}
```

```
import { memo } from "react";

export const CounterChild = memo((props)=>{
   // child component
   console.log("Counter child component rendered");
   const { plusClickHandler } = props;
    return (
        <div>
            <button onClick={plusClickHandler}>+</button>
```

```
        </div>
    )
});
```